

CSCE 701 CYBERSECURITY · MODULE 05 · LECTURE NOTES

Platform and Isolation

What the operating system and the runtime actually separate.

MODULE OUTCOMES

Explain the isolation mechanisms and their boundaries.

Compare processes, containers, and virtual machines as boundaries.

Explain privilege separation and sandboxing.

Explain the memory-safety classes at a defensive level.

Choose an isolation boundary proportionate to the threat.

1 The boundaries

Boundary	Enforced by	Escaped via
Same process, different modules	Nothing — language and library conventions only.	Any memory-safety bug, any reflection, any shared mutable state. This is not a security boundary and should never be described as one.
Separate processes	The memory management unit and the kernel.	Kernel vulnerabilities, shared files, shared sockets, permissive IPC, and the filesystem. A real boundary, with a large surface.
Container	Kernel namespaces and control groups — <i>one shared kernel</i> .	Kernel vulnerabilities (which are shared), and misconfiguration — a privileged container, a mounted socket, or an over-broad capability set.
Virtual machine	The hypervisor — <i>separate kernels</i> .	Hypervisor vulnerabilities (a far smaller surface than a kernel) and side channels (§4's callout).
Separate physical machine	Physics.	The network, and whatever authorisation you granted (Modules 03 and 04).

A BOUNDARY IS ONLY AS STRONG AS WHAT ENFORCES IT

'Isolated' is not a property; it means 'isolated by a specific mechanism against a specific class of attack' — and asking which mechanism and which class is the whole analysis.

So a container isolates a buggy process from another buggy process rather well, and isolates deliberately hostile code from your kernel considerably less well, because they share one kernel and a kernel's attack surface is enormous — millions of lines, hundreds of system calls, and a steady stream of privilege escalation vulnerabilities.

Which is why running untrusted code requires a stronger boundary — a virtual machine, a microVM with a deliberately minimal device model, or a language-level sandbox with a small, auditable interface (WebAssembly with a restricted host interface is the current example).

And it is why 'we run it in a container' is not an answer to 'how do you isolate customer code' — it is an answer to 'how do you stop deployments interfering with each other', which is a legitimate and completely different question. Conflating packaging isolation with security isolation is the commonest error in this module.

2 Privilege separation

THE IDEA: split a program so that the part handling untrusted input has the LEAST privilege, and the privileged part has the SMALLEST interface.

THE CANONICAL EXAMPLE -- a network daemon:

- a small PRIVILEGED PARENT: binds the port, opens log files, holds the host key, and answers a handful of specific, well-defined requests
- an UNPRIVILEGED CHILD per connection: parses the protocol, handles all the untrusted bytes, runs in a sandbox with no filesystem and no network access

they communicate over a NARROW, WELL-DEFINED channel

SO A PARSER BUG YIELDS AN UNPRIVILEGED SANDBOXED PROCESS, rather than root.

THE DESIGN RULES

- the privileged part must be SMALL enough to review line by line
- the interface between the two must be NARROW enough to reason about completely, and must validate everything crossing it
- the unprivileged part must hold NOTHING worth stealing -- no keys, no other users' data
- and privileges must be dropped EARLY and IRREVERSIBLY

AND THE SAME SHAPE APPLIES WELL ABOVE THE OPERATING SYSTEM: a browser's renderer process per site, a microservice per trust level, a separate process for decoding untrusted media.

The parser is where untrusted input is handled, so the parser is what to isolate — which is the single most transferable instance of this design, and it applies to image decoders, document parsers, protocol handlers, and template engines alike. **A browser's site-isolated renderer is exactly this pattern at a different scale**, and it is why a rendering engine vulnerability is now a sandbox escape chain rather than a single bug.

3 Memory safety

MEMORY-UNSAFE LANGUAGES PRODUCE A PREDICTABLE CLASS OF VULNERABILITY

Out-of-bounds read and write, use after free, double free, uninitialised memory, and integer overflow leading to a too-small allocation — a small and very well-understood set of root causes.

And they have accounted for the substantial majority of severe vulnerabilities in large C and C++ codebases for decades, which the major vendors' own published analyses agree on (Microsoft and Chromium have both reported around 70%).

The platform mitigations help and do not solve it: stack canaries, address space layout randomisation, and non-executable memory each remove one exploitation *technique* and leave the underlying bug in place — which is why each was followed by a new technique rather than by the end of the class.

So the structural answers are a memory-safe language for new code (Rust, Go, Java, C#, and anything managed) **and sanitisers plus fuzzing for existing code** (CSCE 713's subject) — **and the language choice is the intervention with a large, measured effect**, which is why national cybersecurity agencies now recommend it in writing.

- **Non-executable data pages. Removes injecting and jumping to new code** — and reuse-of-existing-code techniques work around it entirely, which is why it was not the end of the problem.
- **Address space layout randomisation. Requires the attacker to learn an address before they can use one**, so it raises the cost — **and it is defeated by any information leak**, which is why leaks are now valuable in their own right.
- **Stack canaries. Detect sequential overwrites past a stack buffer**, and **not targeted writes, heap corruption, or anything that does not cross the canary**.
- **Control-flow integrity and hardware shadow stacks. Restrict where execution may be redirected to a set of legitimate targets** — **currently the most effective of these mitigations**, and the one with hardware support arriving.
- **And compiler hardening flags, which are close to free. Fortified standard library calls, bounds-checked containers in debug and release, integer overflow trapping, and stack protection cost a few percent and are frequently off** — **turn on everything the toolchain offers and measure the cost**, because the defaults are historical rather than considered.

4 Choosing a boundary

What you are running	Use at least
Your own code, as separate services	Separate processes; containers for packaging and resource limits. The threat is bugs, not malice.
Your own code handling untrusted input	Privilege separation plus a sandbox for the parsing part (§2).

What you are running	Use at least
A third-party library you cannot audit	Process isolation, and a sandbox if it parses anything — which is Module 08's dependency risk made architectural.
Another tenant's code	A virtual machine or a microVM. <i>Not a container.</i> See the note.
Deliberately hostile code — malware analysis, untrusted submissions	A virtual machine, on separate hardware, with no network access and a snapshot to revert to.

The fourth row is the one organisations get wrong. Container-level isolation between tenants has been the root cause of real multi-tenant escape incidents, and microVM technologies exist precisely because of it — they provide VM-grade isolation at container-grade startup cost, which is what the workload actually needs.

AND SIDE CHANNELS CROSS EVERY BOUNDARY IN THE TABLE

Shared caches, shared branch predictors, shared memory bandwidth, shared execution ports, and even power consumption leak information across process, container, and virtual machine boundaries — because those resources are shared at a level below the one the boundary is enforced at.

Which the Spectre and Meltdown family demonstrated at scale — the architectural isolation was entirely intact and the microarchitectural state was shared, and the boundary was defined over the former.

And the mitigations are expensive: cache partitioning, disabling or flushing shared predictors on context switch, and refusing to co-schedule different tenants on sibling hyperthreads of the same physical core all cost real and sometimes substantial performance (CSCE 614's subject).

So the honest position is that if you share hardware with an adversary, some information leakage is likely — which is why the highest-sensitivity workloads run on dedicated hardware, and that is a proportionality judgement rather than paranoia (Module 04 §4's reasoning, applied to the platform).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google — Building Secure and Reliable Systems, the isolation chapters (free PDF)	§1 and §2 from people operating this at scale, free in full.
Provos, Friedl & Honeyman — Preventing Privilege Escalation (free)	§2's design, in the paper that introduced it to OpenSSH — and the design rules are stated explicitly.
Szekeres et al. — SoK: Eternal War in Memory (free)	§3's classes and mitigations, systematised — which attack each mitigation stops and which it does not.

Resource	What it covers for this module
Kocher et al. — Spectre Attacks (free)	§4's callout — and worth reading for the argument that the boundary was defined at the wrong level.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **List every isolation boundary in a system you operate** and name what enforces each.
- 2 **Find one described as a security boundary that is not one**, and say what it actually provides.
- 3 **Inspect a container's capabilities and mounts** and identify what would weaken the boundary.
- 4 **Privilege-separate a small program of your own** that parses input, and verify the parser cannot touch the filesystem.
- 5 **Narrow the interface between the two parts** and document what crosses it.
- 6 **Compile a C program with and without hardening flags** and measure the performance difference.
- 7 **Check which mitigations are enabled** in a binary you ship.
- 8 **Run a sanitiser over an existing C or C++ codebase** and report what it finds.
- 9 **Classify five workloads** against §4's table and say whether the current boundary is adequate.
- 10 **Read a side-channel paper** and state which boundary it crossed and what was shared.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five isolation boundaries, what enforces each, and how each is escaped.
- 2 What is the difference between a container and a VM, and what does it decide?
- 3 Why is 'isolated' not a property?
- 4 Why is 'we run it in a container' not an answer for customer code?
- 5 Give the privilege separation design and its four rules.
- 6 Why is the parser the thing to isolate?
- 7 Name the memory-safety classes and what fraction of severe vulnerabilities they are.
- 8 Name five mitigations and what each does not stop.
- 9 Give the boundary to use for five workloads, and the row most often got wrong.
- 10 Why do side channels cross every boundary, and what follows?